

FPGA Implementation of ANN Using Customized Instruction Set for 1-D Signal Processing

Anudita Bagal

Electronics and Telecommunication Engineering,
Pimpri-Chinchwad College of Engineering Pune,
Pune, India
anudita.bagal20@pccoepune.org

Dr. Prof. Varsha Harpale

Electronics and Telecommunication Engineering,
Pimpri-Chinchwad College of Engineering Pune,
Pune, India
varsha.harpale@pccoepune.org

Abstract- This work describes the implementation of hardware architecture based on ANN and its hardware realization using hardware description language (Verilog) on FPGA. The objective of this project is "FPGA implementation of ANN using customized Instruction set for 1-D signal processing". Our aim in this project is on designing small in size and functional processor architecture. The proposed project presents architectural design and execution of artificial neural network processor with a customized instruction set processor for 1 D signal processing. The ANN processor architecture is flexible, supporting random counts of layers and a number of 784 artificial neurons (ANs). The proposed architecture will support the feed-forward algorithm. The proposed architecture is flexible with the counts of input data; output data and signals will represent the fixed-point format and will compare the accuracies of the Tanh function and sigmoid function.

Keywords: Artificial Neural Network, Activation Functions, Back propagation algorithm, Python, Verilog.

I. INTRODUCTION

A human brain has several relative parts known as neurons. Every of them is connected to thousands of different neurons. They convey by causing messages within the sort of Electrical pulse signals. A human-made neural network (ANN) is an ideal scientific pattern that is inspired by the method biological brain processes data. An ANN consists of interconnected processed data referred to as artificial neurons (ANs), which loosely model the neurons in a biological brain. An ANN consists of interconnected process parts cited as artificial neurons (ANs).that loosely model the interconnected ANs area unit organized among input, hidden, and output layers. An ANN stores representations within the interconnections between ANs (like the junction in the biological brain), every that holds a value noted as the weight. Equally to biological brains, ANNs determined examples. An ANN is organized for a particular application through learning techniques. The learning techniques involve changes to the weights of the interconnects supported by the input patterns. Figure 1. Therefore, rather than being programmed as in microprocessors, ANNs determine what weights to take by a process called training. After classifying enough examples, neural networks will classify new objects they have never observed before and gives predictive information. Throughout the operation, a pattern is applied to the input layer. Every A responds to the input data. Using a set of weighted interconnects, certain ANs in the network respond to the well-built when they sense an identical pattern.

The response is relayed to the other ANs within the hidden, The result is generated in the output layer

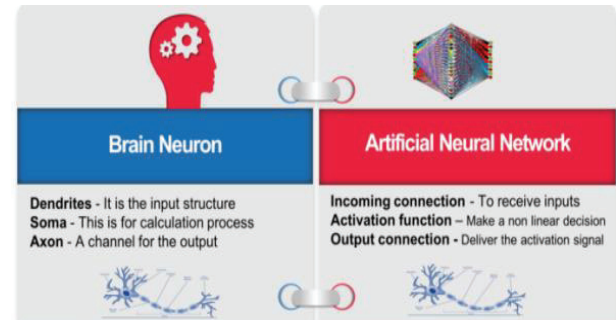


Fig. 1. Biological brain and Artificial neural network

This paper demonstrates FPGA Implementation of ANN Using Customized Instruction Set for 1-D Signal Processing, the paper is organized as below section II represents literature survey, proposed methodology is explained in section III,description of design and implementation in section IV ,Experimentation results in section IV ,and conclusion present In section V.

II. LITERATURE SURVEY

[1] In 2018, A paper presented by Daniel Valencia and his team at San Diego state university designed an ANN-based processor which consumes significantly less power and is small in size.

[2] In 2018, Ziyi Tian from Beijing University shows comparing the accuracy of three CNN-based models based on layers and find the highest number of layers is the most accurate.

III. METHODOLOGY

A. Dataset

We have used two datasets one for hardware and another one for ECG sampling.

- 1) The database system for handwritten data is MNIST which is one of the largest database systems. The Application of the database is training and AI-based project
- 2) We have used here MIT-BIH Atrial Fibrillation database for ECG voice samples, This dataset is described by manual identification of ECG beats.

B. Neurons

A human brain has several relative parts known as neurons. Each of them is connected to thousands of different neurons. They convey by causing messages within the sort of Electrical pulse signals. Figure 2

$$y_n = \sum_{i=1}^{M_n} [w_{in}x_i] + b_n,$$

Where w_{in} is the weight between inter-connected layers, M_n is the number of inputs of the n th layer b_n is the bias of n -th AN.

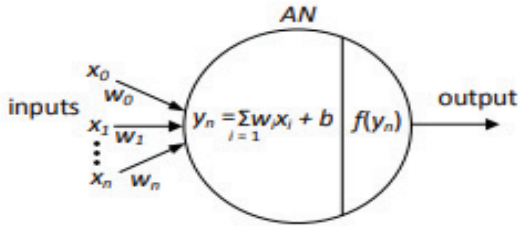


Fig. 2. Artificial neuron architecture

C. Activation function

The result of ANN is based on the activation function Fig 3 , because they can give us pre-calculated values and store them in a ROM. these ROM are also called lookup tables (LUT,s) these LUTs ROM are built either using block RAMs or Distributed RAMs (LUTs and FFs). In ANN activation function plays a significant role, activation of the neuron is dependent upon activation function. The main functionality of the activation function is to obtain output from input values and sends it to the various layers. We have used two activation functions here one is sigmoid having a range between 0 to 1 and the other one is Tanh whose range lies between -1 to 1 vice versa. Figure 4. We will also compare the accuracy of both activation functions

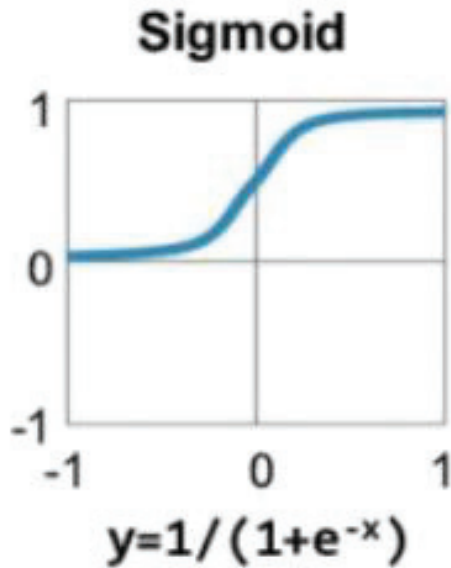


Fig. 3. Sigmoid function

Hyperbolic Tangent

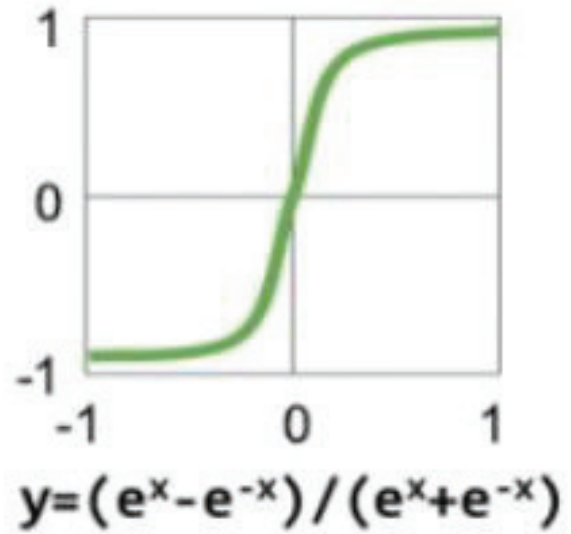


Fig. 4. Tanh function

D. The algorithm used

Feedforward Algorithms are the combination of many layers it is also called the perceptron, Decision flow of FNN is in one direction only (never flows backward) and connected by the nodes to nodes, Inputs are passed to the next layer by this algorithm, and this algorithm is responsible for passing inputs neurons Figure 5 to following layers.

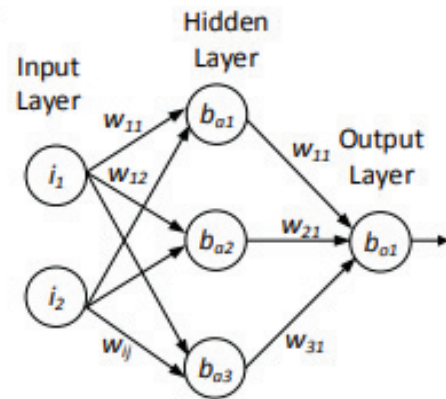


Fig. 5. Feed forward Neural Network)

IV. DESIGN & IMPLEMENTATION

Step to design ANN Architecture: Steps to design proposed ANN processor:

- Designing a basic neuron.
- Activation function design
- Compare the Accuracy of Tan h and sigmoid)
- Designing the layers.
- Hardware wrap-up
- Verification.

1) First we should have to perform training of neurons because in hardware training of neuron is quite a difficult task, (neurons training needs innumerable digital circuitry) we have used here pretrained neuron in software be Training. Feedforward algorithms do not support hardware very much. Networks are trained at one time only it suggests such a waste of resources used for training. For most applications, training is not the time-critical path, once we translate it to actual neural network implementation what we are looking for is real-time performance like detection and classification .

So we will train neurons using a machine learning library like tensor flow and, PyTorch, etc., we have to fix the weights and biases values and will use them during hardware design directly.

2). Number Representation: (Weights are biases can be positive or negative) here we are using 2's complement representation instead of sign-magnitude representation. the reason behind using 2's complement representation is because Addition and subtraction are more efficient in 2's complement.

Here we will go with Fixed-point representation to opt-out of floating-point representation is maybe critical Floating-point representation helps in representing large numbers and they may provide better precision but their implementation and manipulation are quite hard and needs many resources so we won't able to implement more than a few tens of neurons on a platform like Zynq board. so we have used fixed-point representation since the input values used in NNs are generally normalized (between 0 to 1 or -1 to 1) so there won't be an issue of not being able to represent large numbers.

3). Fully connected neural network (every neuron is fully connected with every layer, feed-forward neural network)

A feed-forward neural network is a type of artificial neural network in which nodes' connections do not form a loop. Often referred to as a multi-layered network of neurons, feedforward neural networks are so named because all information flows in a forward manner only. Input neurons pass into input nodes and hidden layers then lastly go to output, network layer doesn't allow to flow of data to reverse.

The purpose of feed-forward neural networks is to approximate functions. The components of the feed-forward neural network are the input layer, hidden layer, and output layer. The input layer contains the neuron and the input layer data is passed to the next layer or tier, the input layer's total number of neurons is equal to the number of variables in the dataset. The hidden layer is the intermediate layer, which is concealed between the input layer and output layer. This layer has a large number of neurons that perform alterations on the inputs. They then communicate with the output layer considering model structure we can get desired results by the output layer.

The main components of general architecture (Figure 6) are going to use memory called a weight memory for storing the weight, the depth of the memory will depend upon the number of inputs coming to the neuron so each location in the memory will represent a weight value corresponding to that particular input now this we can implement either as a ROM or as a RAM that means again as we mentioned before we have to use pre-trained network so the weight values will be

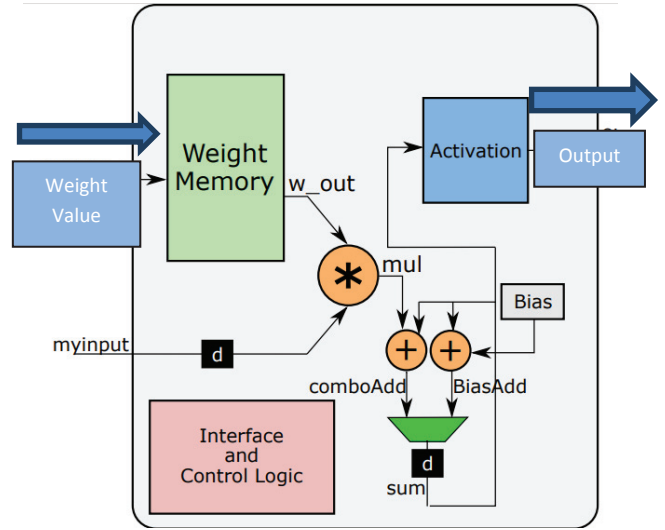


Fig. 6. Architecture of proposed ANN

already available so we can just store it and we can store the weight values Here and later if we want to change we will be able to do it for flexibility so we will do both so both will be supported in our design. the multiplication we are multiplying each weight with the Corresponding input or each input with the corresponding weight and that organized by this idea there will be one who will get the appropriate weight from the memory and multiplied by the input we have then an operation base on the accumulator will be performed which is here so it will add This output with the previous output. Finally this multiplied and accumulated value will get added to this bias also through this adder and then it will be go to the activation function .In ROM where we store our pre calculated value for our corresponding activation function for a sigmoid. Training of neuron pre trained neuron in software because training needs plenty numbers of hardware .Feed forward algorithms are not very much hardware friendly; most networks are trained only one time which means a waste of resources. For most applications, training is not the time-critical path, once we translate it to actual neural network implementation what we are looking for real time performance like detection and classification so we will train in Python and calculate the weights and biases value and directly use them while we make the software-hardware design.

4). Represent numbers in hardware: (Weights are biases can be positive or negative) here we are using 2's complement representation is instead of sign-magnitude representation.

The reason behind using 2's complement representation is because Addition and subtraction are more efficient in 2's complement. Fixed point representations. (there are two types of number representation: Floating-point representation helps in representing large numbers and they may provide better precision but their implementation and manipulation are difficult. They consume so many resources so we won't able to implement more than a few tens of neurons on a platform like the Zynq board. So we have used fixed-point representation since the input values used in NNs are generally normalized (between 0 to 1 or -1 to 1) so there won't be an issue of not being able to represent large numbers.

5). Activation functions: Many neural network uses non - linear functions such as sigmoid as activation function,

Because they can give us pre-calculated values and store them in a ROM. these ROM are also called lookup tables (LUT, s) these LUTs ROM are built either using block RAMs or Distributed RAMs (LUTs and FFs).

6). Using IP cores: We generally won't instantiate XILINX IP cores (RAMs, DSP slice, etc.) when building a neural network this will bring more flexibility to the design

If we are using IP cores for example to design a ROM for activation function and if we decide to change the size of ROM, we won't be able to do this by changing a parameter,

We will have to do it through the XILINX IP core generator. But IP cores are highly efficient in implementation because Block Ram is much more efficient than distributed RAMs when making ROMs. Hence we will use appropriate method instead of directly instantiate them.

V. RESULT & ANALYSIS

```
Random weights at the start of training
[[-0.16595599]
 [ 0.44064899]
 [-0.99977125]]
New weights after training
[[5.39428067]
 [0.19482422]
 [0.34317086]]
Testing network on new examples ->
[0.99995873]

Process finished with exit code 0
```

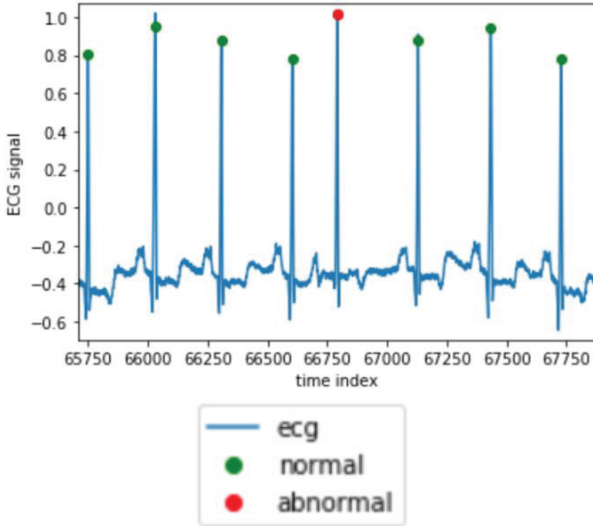


Fig. 7. ECG signal plot for normal and abnormal sample

TABLE I. TESTING OF VARIOUS MODELS:

Models	Accuracy	AUC	Recall	Precision
a).CNN	0.972	0.992	0.938	0.987
b).Dense(NN)	0.839	0.907	0.671	0.849
c).RNN (LSTM)	0.714	0.872	0.936	0.642

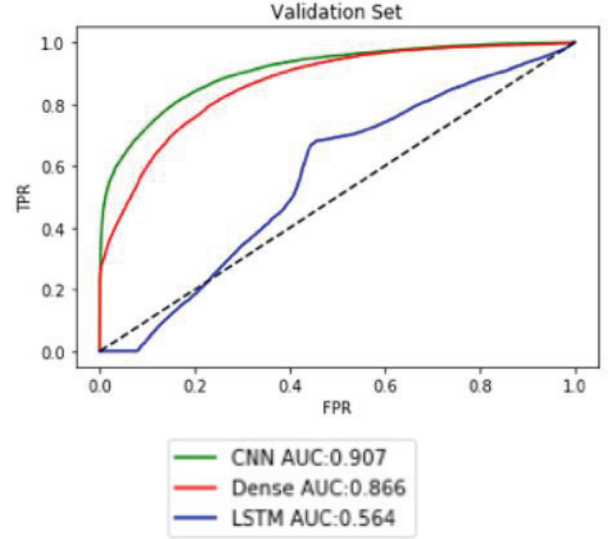


Fig. 8. Testing Accuracies between different models

VI. CONCLUSION

The proposed ANN Architecture and the hardware supports ANNs with dictatorial counts of layers, numbers of ANs per layer, random interconnect structures, and a feed-forward network.

The proposed ANN Architecture for 1 D signal processing will support the time series data. This article presents the Artificial neural networks .FPGA implementation for 1-D signal processing, Application of 1-D signal is time-series data, we have taken ECG data samples of 1000 patients for showing their output is normal or abnormal, on comparing these three types of deep learning Models (CNN, Dense Auc, and LSTM), we obtained the accuracies CNN model (97.2) have most accurate than Dense Auc(83.9) and LSTM(71.4).

Application of Proposed Architecture is used in a compact area- and less power-consumed embedded applications utilizing relatively-sized ANNs.

REFERENCES

- [1] Daniel Valencia, "An Artificial Neural Network Processor with a Custom Instruction Set Architecture for Embedded Applications" "Department of Electrical and Computer Engineering San Diego State University, San Diego, U.S.A., 2020.
- [2] Ziyi Tian" Deep Learning for Electrocardiogram (ECG) Identification" B.Eng., Beijing University of Technology, 2018
- [3] D. Valencia, J. Thies, and A. Alimohammad, "Frameworks for efficient brain-computer interfacing," IEEE Transactions on Biomedical Circuits and Systems, vol. 13, no. 6, pp. 1714–1722, 2019.
- [4] Bhaskar Bateja, Pankaj Sharma "FPGA implementation of artificial neural networks", NIT Rourkela, 2007